Risk-Aware Action Selection for Uncertainty-Calibrated Tool-Using Language Model Agents

Akhilesh Varma
akhverm@gmail.com

September 2026

Abstract

Tool-using language-model agents can solve multi-step tasks by interleaving reasoning, information retrieval, and external actions. However, a locally plausible action can create a globally invalid trajectory when the model selects an unsuitable tool, supplies malformed arguments, trusts a misleading observation, or continues after uncertainty has become decision-relevant. This paper proposes an uncertainty-calibrated, risk-aware control architecture for reliable multi-step execution. The architecture combines a planner, a multi-signal uncertainty estimator, a logical–syntactic–semantic verifier, and a controller that selects among execution, retrieval, replanning, clarification, and human escalation. We formalize action uncertainty, trajectory success, selective abstention, cost-sensitive utility, and tail risk. We derive bounds showing how per-step calibration errors accumulate, establish sufficient conditions under which verification improves trajectory success, and characterize when clarification dominates autonomous action. We further provide an implementable pseudocode procedure and an evaluation protocol spanning interactive web, tool-use, conversational API, and software-engineering benchmarks. The central claim is intentionally prospective: the proposed composition is a testable research hypothesis, not an assertion of completed empirical validation.

Keywords: language-model agents; tool use; uncertainty calibration; selective prediction; verification; risk-sensitive control; human-in-the-loop systems

1 Introduction

Large language models (LLMs) have moved from producing isolated text to operating as controllers that select tools, interpret observations, and execute multi-step plans. ReAct demonstrated the value of interleaving reasoning traces with environment actions and observations [1]. Toolformer showed that a language model can learn when and how to call simple APIs through self-supervised filtering of candidate calls [2]. These results establish the usefulness of tool-mediated computation, but they do not remove a basic reliability problem: an agent may be confidently wrong about the next action.

The problem is amplified by trajectory length. If a task requires T actions and action i fails with probability p_i, then under conditional independence the probability of an entirely successful trajectory is

$$\mathbb{P}(\text{success}) = \prod_{i=1}^{T}(1 - p_i). \tag{1}$$

Even when every p_i is small, the product can decrease rapidly with T. More generally, without independence, the union bound gives

$$\mathbb{P}(\text{at least one failure}) \leq \sum_{i=1}^{T} p_i, \tag{2}$$

which still makes clear that local reliability must improve as horizons grow.

Interactive benchmarks expose this gap. WebArena reports a 14.41% end-to-end success rate for its best GPT-4-based agent compared with 78.24% for human participants [7]. τ-bench reports that GPT-4o succeeds on fewer than half of its tasks and that repeated-trial reliability in retail is below 25% at pass^8 [8]. SWE-bench similarly presents repository-level tasks whose original evaluation found that Claude 2 solved only 1.96% of 2,294 real GitHub issues [9]. These results are benchmark-specific, but they motivate a common design requirement: the agent should know when its next action is not sufficiently trustworthy.

This paper asks: *How can an agent estimate whether it is likely to be correct before taking the next tool action, and use that estimate to execute, verify, retrieve, clarify, or defer?* We make four contributions:

1. We define an architecture that separates planning, uncertainty estimation, verification, and risk-aware control.

2. We introduce a composite uncertainty estimator that combines reasoning-path agreement, tool-result consistency, retrieval confidence, and action-validity checks.

3. We state and prove propositions concerning calibration error accumulation, verification gains, and clarification decisions.

4. We specify an evaluation methodology with baselines, metrics, ablations, adversarial tool conditions, and repeated-trial reliability tests.

2 Related Work and Research Gap

2.1 Reasoning–acting agents

ReAct established a prompting pattern in which reasoning traces and environment actions are interleaved, allowing observations to revise an evolving plan [1]. Toolformer extended this direction by learning API-call placement and arguments through self-supervised filtering of candidate calls [2]. ToolLLM and ToolBench broadened the tool-use setting to thousands of real-world APIs and automatic evaluation [10]. These systems primarily optimize whether an agent can produce a useful trajectory. The present work focuses on a complementary question: whether the agent can estimate the conditional risk of its next action before execution.

2.2 Calibration, abstention, and verification

Calibration asks whether predicted probabilities correspond to empirical frequencies, whereas selective prediction asks when a model should abstain to control risk at a chosen coverage [3, 4]. This distinction is important for agents because a high-confidence language-model completion is not necessarily a calibrated probability of tool success. Self-consistency improves reasoning by aggregating sampled paths [5], and outcome or process verifiers can rank candidate solutions [13, 14]. However, agreement and verifier scores can be correlated with the same underlying error. The proposed estimator therefore treats them as features and evaluates calibration independently.

2.3 Risk-sensitive sequential control

Expected utility alone can select an action with a small probability of catastrophic loss. Risk-sensitive Markov decision process formulations, including conditional value-at-risk (CVaR), explicitly penalize adverse tails [6, 15]. The proposed controller adapts this idea to language-agent actions by assigning harm weights and allowing information-gathering actions to change future risk. This creates a research gap at the intersection of calibrated selective prediction

Table 1: Positioning of the proposed framework relative to representative approaches.

Approach	**Primary strength**	**Unresolved reliability issue addressed here**
ReAct	Interleaves reasoning, action, and observation	Does not by itself calibrate next-action failure risk or select abstention thresholds.
Toolformer / ToolBench	Learns or evaluates API use at scale	Tool selection and argument validity can remain overconfident under novel or noisy APIs.
Self-consistency	Aggregates multiple reasoning paths	Agreement can be correlated and is not a calibrated probability of correctness.
Outcome/process verifiers	Scores candidate solutions or intermediate steps	Verifier errors, distribution shift, and tool-side effects require explicit risk control.
Selective prediction	Controls risk by abstaining at a chosen coverage	Classical formulations do not model long-horizon tool trajectories and action costs.
CVaR control	Penalizes low-probability, high-loss outcomes	Requires a calibrated loss distribution and an action-level uncertainty interface.

and tool-mediated sequential control: existing components are individually studied, but their end-to-end composition for action-level reliability remains an empirical hypothesis.

3 Problem Formulation

Let a task be represented by an initial state s_0, a natural-language goal g, a tool set $\mathcal{T}$, and a trajectory horizon T. At time t, the agent observes history

$$h_t = (g, s_0, a_0, o_1, a_1, o_2, \ldots, a_{t-1}, o_t), \tag{3}$$

where $a_t \in \mathcal{A}(h_t)$ is a candidate action and o_{t+1} is the resulting observation. An action may be a tool invocation, an information-retrieval request, a clarification question, a re-planning operation, or an escalation request.

Each candidate action has an unobserved binary outcome variable $Y_t(a) \in \{0, 1\}$, where $Y_t(a) = 1$ means that the action is correct, policy-compliant, and safe relative to the current state and task. The conditional action failure probability is

$$p_t(a) = \mathbb{P}\left(Y_t(a) = 0 \mid h_t, a\right). \tag{4}$$

The agent produces an estimate $\widehat{p}_t(a)$ before executing the action. The key calibration requirement is that, for actions receiving a predicted risk near q, the empirical failure frequency should also be near q.

Definition 1 (Action calibration)**.** *An estimator $\widehat{p}$ is ε-calibrated on a deployment distribution if for every measurable bin $B \subseteq [0, 1]$ with nonzero probability,*

$$|\mathbb{E}[\mathbf{1}\{Y = 0\} \mid \widehat{p} \in B] - \mathbb{E}[\widehat{p} \mid \widehat{p} \in B]| \leq \varepsilon. \tag{5}$$

Calibration is distinct from accuracy. Guo et al. showed that modern neural networks may be poorly calibrated despite strong predictive performance and found temperature scaling effective in their image and document-classification experiments [3]. The proposed agent therefore treats uncertainty as a separately measured object rather than equating language-model confidence with correctness.

4 Sequential Decision Model

The agent can be modeled as a partially observed, constrained decision process. Let $s_t \in \mathcal{S}$ be the latent environment state, $b_t(s) = \mathbb{P}(s_t = s \mid h_t)$ the agent's belief state, and $\mathcal{O}$ the observation space. A tool action a_t induces a transition kernel $P(s_{t+1} \mid s_t, a_t)$ and an observation kernel $O(o_{t+1} \mid s_{t+1}, a_t)$. The policy is therefore a mapping

$$\pi(a_t \mid h_t) = \pi(a_t \mid b_t, g, \mathcal{T}). \tag{6}$$

The belief update is

$$b_{t+1}(s') = \frac{O(o_{t+1} \mid s', a_t) \sum_s P(s' \mid s, a_t) b_t(s)}{\sum_{\bar{s}} O(o_{t+1} \mid \bar{s}, a_t) \sum_s P(\bar{s} \mid s, a_t) b_t(s)}. \tag{7}$$

In practice, the exact state and transition models are unavailable. The proposed uncertainty estimator approximates the effect of belief uncertainty using observable agreement, retrieval, schema, and verification features. This makes the architecture applicable to black-box APIs while retaining a principled sequential interpretation.

Define a constrained action set

$$\mathcal{A}_{\text{safe}}(h_t) = \{a \in \mathcal{A}(h_t) : V_t^{\text{syn}}(a) V_t^{\text{pol}}(a) = 1\}. \tag{8}$$

Hard constraints are enforced before utility optimization. This separation prevents a high expected reward from compensating for an invalid argument, an unauthorized operation, or an irreversible action outside the agent's authority.

For information-gathering actions q, define the expected value of information (EVI) as

$$\text{EVI}(q \mid h_t) = \mathbb{E}_{o \sim O(\cdot \mid h_t, q)} \left[\max_{a \in \mathcal{A}_{\text{safe}}(h_t, o)} J(a \mid h_t, o) \right] - \max_{a \in \mathcal{A}_{\text{safe}}(h_t)} J(a \mid h_t) - C(q), \tag{9}$$

where J is the risk-adjusted objective. Retrieval is selected only when $\text{EVI}(q \mid h_t) > 0$, subject to a latency or budget constraint. Clarification is a special information action whose observation is the user's response.

4.1 Threat model

The reliability analysis considers five failure sources: (i) model hallucination, where the plan or argument is unsupported; (ii) schema failure, where an invocation is syntactically invalid; (iii) semantic mismatch, where a valid invocation does not satisfy the goal; (iv) observation corruption, where a tool returns stale, misleading, or adversarial information; and (v) compounding state error, where an early failure changes later preconditions. A robust evaluation must vary these factors independently. The method does not assume that external tools are truthful; it explicitly measures disagreement and result consistency.

5 Architecture

Figure 1 shows the proposed control loop. The planner generates a structured action proposal. The estimator converts heterogeneous evidence into a calibrated risk estimate. The verifier checks validity before execution. The controller optimizes expected utility subject to risk and cost preferences.

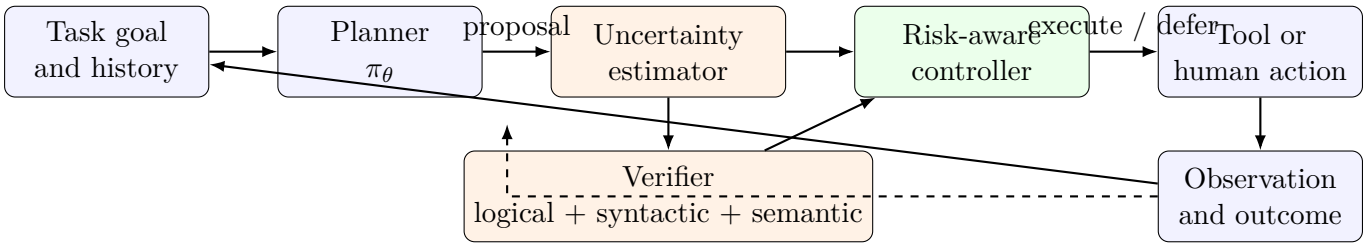

Figure 1: The proposed uncertainty-calibrated control architecture. The dashed feedback path supplies post-action evidence for calibration and future risk estimation.

5.1 Planner

The planner produces a set of candidate actions rather than a single irreversible command:

$$\mathcal{P}_t = \{(a_{t,j}, \rho_{t,j}, \xi_{t,j})\}_{j=1}^{K}, \tag{10}$$

where $a_{t,j}$ is an action, $\rho_{t,j}$ is a rationale or plan fragment, and $\xi_{t,j}$ contains explicit preconditions, expected observations, required arguments, and possible failure modes. A structured proposal makes it possible to verify arguments independently of free-form generation.

5.2 Multi-signal uncertainty estimator

For each action a, define four normalized signals in $[0, 1]$:

$$q_{\text{agree}}(a) = 1 - \frac{1}{M} \sum_{m=1}^{M} \mathbf{1}[\tilde{a}_m = a], \tag{11}$$

$$q_{\text{tool}}(a) = 1 - \text{Cons}(o^{(1)}, \ldots, o^{(L)}), \tag{12}$$

$$q_{\text{retr}}(a) = 1 - r(a), \tag{13}$$

$$q_{\text{valid}}(a) = 1 - v(a). \tag{14}$$

Here $\tilde{a}_m$ is the action proposed by reasoning path m, Cons measures agreement among independent or perturbed tool-result checks, $r(a)$ is retrieval confidence, and $v(a)$ is the verifier's validity score. A feature vector is

$$\phi_t(a) = [q_{\text{agree}}(a), q_{\text{tool}}(a), q_{\text{retr}}(a), q_{\text{valid}}(a), d_t, \ell_t, c_t], \tag{15}$$

where d_t is estimated horizon remaining, ℓ_t is argument complexity, and c_t is action cost.

A practical risk head is a monotone logistic model followed by post-hoc calibration:

$$z_t(a) = \beta_0 + \boldsymbol{\beta}^\top \phi_t(a), \tag{16}$$

$$\widetilde{p}_t(a) = \sigma(z_t(a)) = \frac{1}{1 + e^{-z_t(a)}}, \tag{17}$$

$$\widehat{p}_t(a) = \text{Calibrate}(\widetilde{p}_t(a); \mathcal{D}_{\text{cal}}). \tag{18}$$

The calibration set $\mathcal{D}_{\text{cal}}$ must be held out from model fitting and should reflect the deployment task and tool distribution. Isotonic regression, temperature scaling, beta calibration, or conformal selective prediction can be compared empirically.

Reasoning-path agreement is useful but insufficient. Self-consistency improves reported accuracy by voting over sampled reasoning paths [5], yet correlated paths can agree on the same error. Similarly, a verifier score is not a proof unless the verifier itself is formally sound for the action class. The architecture therefore treats these signals as evidence, not guarantees.

5.3 Verifier

The verifier returns a vector

$$V_t(a) = (V_t^{\log}(a), V_t^{\text{syn}}(a), V_t^{\text{sem}}(a), V_t^{\text{pol}}(a)) \in \{0,1\}^4. \tag{19}$$

The components respectively check: (i) logical consistency with the plan and state, (ii) syntactic validity of tool name and arguments, (iii) semantic compatibility between arguments and goal, and (iv) compliance with policy, permissions, and irreversible-action constraints.

For a typed tool schema τ_k, an invocation $a = (k, x)$ passes the syntactic check when $x \in \text{Dom}(\tau_k)$. Semantic validity is represented by a predicate $\mathsf{Sem}(s_t, g, a)$. The verifier can also perform dry runs, query a schema registry, check database invariants, and compare expected and observed effects. For actions that modify external state, a two-phase protocol is preferred: validate first, then commit only after the controller authorizes execution.

5.4 Risk-aware controller

The controller chooses from execution and information-gathering actions using

$$a_t^* = \arg\max_{a \in \mathcal{A}(h_t)} \left[U_t(a) - \lambda R_t(a) - \mu C_t(a) \right], \tag{20}$$

where $U_t(a)$ is expected task utility, $R_t(a)$ is estimated failure or harm risk, $C_t(a)$ is computational, monetary, or latency cost, and $\lambda, \mu \geq 0$ are policy parameters. We define

$$R_t(a) = \widehat{p}_t(a)\, H(a) + \eta\ \text{CVaR}_\alpha\left(L_t(a)\right), \tag{21}$$

where $H(a)$ is a harm severity weight, $L_t(a)$ is a loss distribution induced by uncertain outcomes, η controls tail-risk sensitivity, and CVaR_α is conditional value-at-risk at tail probability α. CVaR provides a principled way to penalize rare but severe outcomes, although its robustness interpretation depends on the underlying decision-process assumptions [6].

The controller exposes five modes:

1. **Execute** when calibrated risk is low and all hard checks pass.
2. **Retrieve** when additional evidence has positive expected value.
3. **Re-plan** when candidate actions are inconsistent, invalid, or dominated.
4. **Clarify** when unresolved user intent contributes more risk than the cost of a question.
5. **Escalate** when risk exceeds a hard threshold or the action is irreversible and outside the agent's authority.

6 Theoretical Analysis

6.1 Trajectory-level calibration error

Assumption 1 (Conditional action calibration)**.** *For each time t, the estimated risk satisfies $|\widehat{p}_t - p_t| \leq \varepsilon_t$ almost surely on the action distribution induced by the controller.*

Proposition 1 (Success degradation under bounded calibration error)**.** *If the conditional independence approximation in eq. (1) holds and Assumption 1 is satisfied, then the difference between true trajectory success and estimated trajectory success is bounded by*

$$\left| \prod_{t=1}^{T} (1 - p_t) - \prod_{t=1}^{T} (1 - \widehat{p}_t) \right| \leq \sum_{t=1}^{T} \varepsilon_t. \tag{22}$$

Proof. Let $x_t = 1 - p_t$ and $y_t = 1 - \widehat{p}_t$. A telescoping expansion gives

$$\prod_{t=1}^{T} x_t - \prod_{t=1}^{T} y_t = \sum_{k=1}^{T} \left(\prod_{t<k} x_t \right) (x_k - y_k) \left(\prod_{t>k} y_t \right). \tag{23}$$

Since $x_t, y_t \in [0,1]$, taking absolute values yields a bound by $\sum_k |x_k - y_k| = \sum_k |p_k - \widehat{p}_k| \leq \sum_k \varepsilon_k$. □ □

Remark 1. *The result is deliberately conservative. It shows why calibration error must be controlled at every step, not only on the final answer. It does not imply that a calibrated estimator is causally correct or robust under distribution shift.*

6.2 Verifier precision and false negatives

The simple verification proposition assumes that verification never rejects a correct action. A more realistic verifier has true-positive rate TPR and false-positive rate FPR, where false positive means accepting an invalid action. Let p be the pre-verification failure probability and let c_v be verification cost. If rejected invalid actions are safely replaced by a fallback with failure probability p_f, then verification improves expected correctness whenever

$$p \ \mathrm{TPR}(1 - p_f) > (1 - p)\,(1 - \mathrm{TPR}) + c_v/H, \tag{24}$$

where H is the expected loss scale. The right-hand side captures missed invalid actions and false rejections of otherwise correct actions. This condition motivates reporting both verifier recall and verifier-induced unnecessary intervention, rather than only the final task-success rate.

Proposition 2 (Log-risk accumulation)**.** *Under the conditional independence approximation and $p_t < 1$, trajectory success satisfies*

$$-\log \mathbb{P}(\text{success}) = \sum_{t=1}^{T} -\log(1 - p_t). \tag{25}$$

Moreover, since $p \leq -\log(1-p) \leq p/(1-p)$ for $p \in [0,1)$,

$$\sum_{t=1}^{T} p_t \leq -\log \mathbb{P}(\text{success}) \leq \sum_{t=1}^{T} \frac{p_t}{1 - p_t}. \tag{26}$$

Proof. The first identity follows by taking negative logarithms of the product in eq. (1). The inequalities follow from the standard bounds $p \leq -\log(1-p) \leq p/(1-p)$ on $[0,1)$. □ □

Remark 2. *The logarithmic form suggests a horizon-aware controller. Two actions with identical immediate risk can have different strategic value when one creates a shorter or longer remaining trajectory. This is why the feature vector includes estimated horizon remaining and why the evaluation should stratify by task length.*

6.3 Conformal and distribution-shift diagnostics

Post-hoc calibration can fail under deployment shift. A practical extension is to maintain a nonconformity score $u_t = \ell(Y_t, \widehat{p}_t)$ on a rolling calibration buffer and monitor its quantiles. If the empirical quantile exceeds a pre-specified tolerance, the controller can raise λ, lower the execution threshold, or require human review. This is a monitoring policy, not a universal distribution-free guarantee, because tool-induced feedback can violate exchangeability. Shift diagnostics should be reported alongside ordinary ECE.

6.4 Verification benefit

Proposition 3 (Sufficient condition for verification to improve success)**.** *Suppose verification transforms each action's conditional failure probability from p_t to p'_t, with $0 \le p'_t \le p_t$ for all t, and with strict inequality for at least one step. Then*

$$\prod_{t=1}^{T}(1-p'_t) > \prod_{t=1}^{T}(1-p_t). \tag{27}$$

after verification, provided verification does not introduce additional failures.

Proof. Each factor $(1-p'_t)$ is at least $(1-p_t)$, and at least one factor is strictly larger. Products of nonnegative factors therefore satisfy the strict inequality. □ □

If verification costs c_v and failure causes expected loss H, verification is beneficial in expected-value terms when

$$H\left[\prod_{t=1}^{T}(1-p'_t) - \prod_{t=1}^{T}(1-p_t)\right] > c_v. \tag{28}$$

This inequality motivates adaptive verification: low-cost, high-impact checks should be applied more aggressively than expensive checks with negligible risk reduction.

6.5 Clarification versus autonomous action

Let a be the best autonomous action and q a clarification action. Let $V(a)$ and $V(q)$ be expected utility, including downstream consequences, and let c_q be the cost of asking. Clarification is preferred whenever

$$V(q) - c_q > V(a). \tag{29}$$

A useful decomposition is

$$V(q) - V(a) = \Delta_{\text{intent}} + \Delta_{\text{risk}} + \Delta_{\text{downstream}}, \tag{30}$$

where the three terms capture reduced ambiguity, avoided failure or harm, and improved future decisions. Thus, a clarification question is rational when the expected value of resolving ambiguity exceeds its interaction cost. Abstention benchmarks show that refusal and calibrated non-answering remain difficult for language models, so this decision should be evaluated directly rather than assumed to emerge automatically [12].

7 Algorithmic Procedure

8 Research Questions and Falsifiable Hypotheses

The framework is designed around falsifiable questions rather than an assumption that every module helps.

1. **RQ1: Calibration.** Does the multi-signal estimator reduce action-level ECE and Brier score relative to raw model confidence, self-consistency frequency, and verifier scores?

2. **RQ2: Reliability.** At matched compute budgets, does risk-aware selection improve task success and pass^k while reducing invalid and catastrophic actions?

3. **RQ3: Efficiency.** Does adaptive verification achieve a better cost–reliability frontier than verifying every action?

Algorithm 1 Uncertainty-Calibrated Risk-Aware Action Selection

Require: Goal g, initial state s_0, tools $\mathcal{T}$, calibration set $\mathcal{D}_{\text{cal}}$, thresholds $\tau_{\text{exec}}, \tau_{\text{esc}}$

$h \leftarrow (g, s_0)$; $t \leftarrow 0$
while not terminal(h) and $t < T_{\max}$ **do**
  $\mathcal{P} \leftarrow \text{Planner}(h, \mathcal{T})$
  for all $a \in \mathcal{P}$ **do**
    $\phi(a) \leftarrow \text{Signals}(a, h)$
    $\widetilde{p}(a) \leftarrow \sigma(\beta_0 + \boldsymbol{\beta}^\top \phi(a))$
    $\widehat{p}(a) \leftarrow \text{Calibrate}(\widetilde{p}(a); \mathcal{D}_{\text{cal}})$
    $V(a) \leftarrow \text{Verify}(a, h)$
    $R(a) \leftarrow \widehat{p}(a) H(a) + \eta \, \text{CVaR}_\alpha(L(a))$
    $J(a) \leftarrow U(a) - \lambda R(a) - \mu C(a)$
  end for
  $a^* \leftarrow \arg\max_{a \in \mathcal{P}} J(a)$
  if hard_violation$(V(a^*))$ or $R(a^*) > \tau_{\text{esc}}$ **then**
    $h \leftarrow \text{EscalateOrAskHuman}(h, a^*)$
  else if $\widehat{p}(a^*) \leq \tau_{\text{exec}}$ and passes$(V(a^*))$ **then**
    $o \leftarrow \text{Execute}(a^*)$; $h \leftarrow \text{Update}(h, a^*, o)$
  else if $\text{EVPI}(h) > \text{CostOfRetrieval}(h)$ **then**
    $o \leftarrow \text{Retrieve}(h)$; $h \leftarrow \text{Update}(h, \text{retrieve}, o)$
  else if $\text{IntentAmbiguity}(h) > \tau_{\text{clarify}}$ **then**
    $u \leftarrow \text{AskUser}(h)$; $h \leftarrow \text{Update}(h, \text{clarify}, u)$
  else
    $h \leftarrow \text{Replan}(h)$
  end if
  $t \leftarrow t + 1$
end while
return FinalAnswerOrEscalation(h)

4. **RQ4: Robustness.** Does the method preserve selective risk under misleading observations, schema perturbations, and distribution shift?

5. **RQ5: Human interaction.** Does uncertainty-triggered clarification reduce downstream failure without producing excessive unnecessary questions?

The primary hypotheses are: H1, calibrated risk estimates improve the ranking of safe versus unsafe actions; H2, adaptive verification improves tail reliability when its expected cost satisfies eq. (28); H3, CVaR-aware control reduces catastrophic-action rate at a tolerable loss in mean utility; and H4, clarification is beneficial mainly on high-ambiguity tasks, not uniformly across all tasks. A null result for any hypothesis is scientifically informative because it identifies a component that does not transfer from benchmark reasoning to tool-mediated control.

8.1 Data splits and leakage controls

Tasks should be divided into training, calibration, development, and held-out test partitions by task template and tool family, not only by random instance. Tool schemas and retrieval corpora in the test partition should include unseen combinations. Calibration labels must be generated from action outcomes that are not used to tune the planner. For software tasks, repository versions and tests must be pinned. For web environments, site snapshots, browser versions, and external knowledge bases must be recorded. These controls prevent a risk head from learning benchmark-specific lexical shortcuts.

Table 2: Recommended minimum reporting bundle.

Report	**Required interpretation**
Task success with confidence interval	Estimate utility, but do not treat it as calibrated reliability.
Action-level ECE, Brier score, reliability diagram	Assess whether predicted risk tracks observed failures.
Pass^k and per-horizon success	Reveal repeated-trial inconsistency and error accumulation.
Invalid-call and catastrophic-action rates	Separate recoverable errors from high-severity failures.
Cost per successful task	Quantify retrieval, verification, clarification, and escalation overhead.
Intervention precision and recall	Measure whether the controller intervenes on genuinely risky actions.
Error taxonomy	Identify which component fails and guide future improvements.

Table 3: Recommended evaluation matrix. The numerical results are intentionally left for future experiments.

Dimension	**Recommended settings**	**Purpose**
Tasks	WebArena, BrowserGym, ToolBench, τ-bench, SWE-bench, API/database environments	Cover browsing, tool selection, user interaction, and code execution.
Baselines	ReAct; self-reflection; majority vote/self-consistency; RAG; verifier-only; proposed controller	Separate planning, retrieval, agreement, verification, and risk-control effects.
Primary metrics	Task success; invalid-call rate; catastrophic-action rate; calibration error; pass^k	Measure both correctness and reliability under repetition.
Efficiency	Steps per success; latency; tokens; tool calls; cost per success	Quantify the price of verification and clarification.
Human factors	Clarification rate; unnecessary clarification; escalation frequency; time to resolution	Test whether uncertainty is actionable rather than merely conservative.
Robustness	Misleading outputs; schema changes; stale retrieval; partial failures; distribution shift	Evaluate failure detection under non-ideal tools.

8.2 Error taxonomy and reporting

Every failed trajectory should be assigned a primary and secondary error category: wrong tool, malformed argument, unsupported claim, stale retrieval, verifier miss, verifier false rejection, state-transition error, unsafe execution, unnecessary clarification, or escalation failure. Report per-category counts and examples. Aggregate success alone can hide a controller that improves task completion by taking more unverified actions or one that appears safe only because it escalates nearly every task.

9 Experimental Methodology

The proposed method should be evaluated as a controller, not only as a language-model prompt. The primary comparison is between otherwise matched agents that differ in their action-selection policy.

9.1 Calibration and reliability metrics

For N executed actions partitioned into B confidence bins, expected calibration error is

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|I_b|}{N} \left|\mathrm{err}(I_b) - \mathrm{risk}(I_b)\right|, \tag{31}$$

where I_b is bin b, $\mathrm{err}(I_b)$ is empirical failure frequency, and $\mathrm{risk}(I_b)$ is average predicted failure probability. Because ECE depends on binning, reliability diagrams, adaptive calibration error, Brier score, and negative log-likelihood should also be reported.

For repeated trials, define

$$\mathrm{pass}^k(x) = \mathbb{P}(\text{all of } k \text{ independent runs solve task } x). \tag{32}$$

This metric is especially relevant for agents whose single-run success conceals inconsistent behavior, as emphasized by τ-bench [8].

9.2 Ablations and statistical protocol

The estimator should be ablated by removing one signal at a time: reasoning agreement, tool-result consistency, retrieval confidence, and verifier output. The controller should then be evaluated with expected utility only, mean-risk penalization, and CVaR-augmented risk. Each configuration should use fixed task seeds and tool versions. Report bootstrap confidence intervals over tasks, paired tests for matched trajectories, and confidence intervals for calibration metrics. Results should be stratified by horizon, action reversibility, tool family, and harm severity.

10 Limitations and Threats to Validity

First, agreement among reasoning paths can be systematically wrong because sampled paths may share a common representation or prompt-induced error. Second, verifier outputs can be overfit, incomplete, or vulnerable to adversarial tool responses. Third, calibration measured on a held-out set can fail after deployment distribution shift, API changes, or changes in the underlying model. Fourth, the conditional independence approximation in eq. (1) is often unrealistic because an early error changes later states and can create correlated failures. Fifth, a risk-averse controller may over-clarify, over-escalate, or refuse useful actions, degrading utility and user experience.

The benchmark ecosystem also has limitations. WebArena is realistic and reproducible but remains bounded by its websites and task suite. ToolBench depends on constructed tool-use data and a fixed API inventory. τ-bench uses simulated users and goal-state matching. SWE-bench relies on repository tests as the principal correctness signal. Consequently, cross-benchmark numerical comparisons should not be interpreted as universal measures of agent competence.

Most importantly, this paper does not claim that uncertainty estimation alone produces safe autonomy. The proposed theory supplies sufficient conditions and measurement requirements. Demonstrating reliable deployment would additionally require independent audits, access-control design, adversarial testing, incident analysis, and domain-specific human governance.

11 Conclusion

This paper proposed a formal architecture for risk-aware action selection in tool-using language-model agents. The architecture separates four responsibilities: planning candidate actions, estimating uncertainty from heterogeneous evidence, verifying logical and executable validity, and selecting between action, information gathering, clarification, and escalation. The analysis

shows that bounded local calibration error can accumulate over long trajectories, that verification is beneficial when its reduction in expected failure exceeds its cost, and that clarification is rational when its information value exceeds the cost of interaction.

The main research opportunity is empirical. A successful system should not merely sound confident or produce plausible traces. It should predict action failure, recognize when its evidence is insufficient, avoid irreversible errors, and remain reliable across repeated trials and changed tool conditions. The proposed evaluation matrix provides a path toward testing those properties without conflating benchmark performance with general safety.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. "ReAct: Synergizing Reasoning and Acting in Language Models." arXiv:2210.03629, 2023. https://arxiv.org/abs/2210.03629.

[2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. "Toolformer: Language Models Can Teach Themselves to Use Tools." arXiv:2302.04761, 2023. https://arxiv.org/abs/2302.04761.

[3] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. "On Calibration of Modern Neural Networks." In *Proceedings of ICML*, 2017. arXiv:1706.04599. https://arxiv.org/abs/1706.04599.

[4] Yonatan Geifman and Ran El-Yaniv. "Selective Classification for Deep Neural Networks." arXiv:1705.08500, 2017. https://arxiv.org/abs/1705.08500.

[5] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." arXiv:2203.11171, 2023. https://arxiv.org/abs/2203.11171.

[6] Yinlam Chow, Mohammad Ghavamzadeh, Lucas Janson, and Marko Pavone. "Risk-Sensitive and Robust Decision-Making: A CVaR Optimization Approach." arXiv:1506.02188, 2015. https://arxiv.org/abs/1506.02188.

[7] Shuyan Zhou et al. "WebArena: A Realistic Web Environment for Building Autonomous Agents." arXiv:2307.13854, 2024. https://arxiv.org/abs/2307.13854.

[8] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." arXiv:2406.12045, 2024. https://arxiv.org/abs/2406.12045.

[9] Carlos E. Jimenez et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" arXiv:2310.06770, 2024. https://arxiv.org/abs/2310.06770.

[10] Yujia Qin et al. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs." arXiv:2307.16789, 2023. https://arxiv.org/abs/2307.16789.

[11] Grégoire Mialon et al. "The BrowserGym Ecosystem for Web-Agent Research." arXiv:2412.05467, 2024. https://arxiv.org/abs/2412.05467.

[12] Facebook Research. "AbstentionBench." arXiv:2506.09038, 2025. https://arxiv.org/abs/2506.09038.

[13] Karl Cobbe et al. “Training Verifiers to Solve Math Word Problems.” arXiv:2110.14168, 2021. https://arxiv.org/abs/2110.14168.

[14] Hunter Lightman et al. “Let’s Verify Step by Step.” arXiv:2305.20050, 2023. https://arxiv.org/abs/2305.20050.

[15] Shun-Pin Lim and Shaojie Malik. “Distributional Bellman Operators for CVaR Optimization.” In *Advances in Neural Information Processing Systems*, 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/c88a2bd0e793550d0e885aa6e31ca277-Abstract-Conference.html.